

Aula 6

DevSecOps: segurança no ciclo de desenvolvimento, pipelines, SAST, DAST, dependências, secrets e shift-left

Material autoral, orientado ao edital retificado do TJSC 2026 e calibrado ao padrão de cobrança da FGV. Foco em teoria aplicável, distinções de prova, exemplos corporativos e questões comentadas.

Disciplina	Segurança da Informação e Governança de TI
Peso no edital	8 questões
Aula	6 de 11
Recorte	Práticas DevSecOps
Estratégia	Acertar distinções entre SAST, DAST, SCA, secrets, pipeline, shift-left e controles de CI/CD

1. Identificação da aula e recorte do edital

Esta é a **Aula 6** da trilha de Segurança da Informação e Governança de TI para o TJSC 2026, cargo Analista de Sistemas, banca FGV. O recorte é: **DevSecOps: segurança no ciclo de desenvolvimento, pipelines, SAST, DAST, dependências, secrets e shift-left**.

A aula cobre diretamente o item **1.8 Práticas DevSecOps** do edital. Como apoio inevitável, usa termos de engenharia de software, integração contínua e entrega contínua, mas sem transformar a aula em engenharia de software genérica. O centro da cobrança é segurança aplicada ao ciclo de desenvolvimento e à esteira de entrega.

A FGV tende a cobrar esse assunto por cenário técnico: uma equipe adota pipeline, scanner, repositório Git, aplicação em execução, dependências, segredos, imagem de contêiner ou API, e o candidato precisa identificar o controle adequado ou o tipo de teste. O erro típico é decorar siglas sem saber em que momento da esteira cada técnica atua.

1.1 O que esta aula entrega

Bloco	Conteúdo	Como a FGV pode cobrar
Fundamentos	DevOps, DevSecOps, SDLC e shift-left	Distinguir cultura, processo e automação de ferramenta isolada

Bloco	Conteúdo	Como a FGV pode cobrar
Pipeline	CI/CD, gates, ambientes, artefatos e credenciais	Identificar riscos da esteira e controles adequados
SAST e DAST	Análise estática e dinâmica de segurança	Diferenciar código-fonte parado de aplicação em execução
SCA	Dependências, bibliotecas, CVE, CVSS e SBOM	Não confundir análise de dependência com análise de código próprio
Secrets	Segredos, vazamento em Git, rotação, vault e menor privilégio	Tratar secret vazado como comprometido, não só apagar commit
Shift-left	Segurança antecipada sem abandonar validação em runtime	Evitar a pegadinha de que shift-left substitui DAST, monitoração ou resposta

2. Siglas e termos essenciais

Sigla/termo	Expansão	Memorização para prova
DevSecOps	Development, Security and Operations	Segurança integrada ao fluxo DevOps, não fase final isolada.
SDLC	Software Development Life Cycle	Ciclo de vida de desenvolvimento de software.
CI	Continuous Integration	Integração contínua: build, testes e validações frequentes após alterações.
CD	Continuous Delivery/Deployment	Entrega contínua ou implantação contínua. Delivery pode exigir aprovação; deployment automatiza até produção.
SAST	Static Application Security Testing	Analisa código, bytecode ou artefatos sem executar a aplicação.
DAST	Dynamic Application Security Testing	Testa a aplicação em execução, de fora para dentro.
IAST	Interactive Application Security Testing	Instrumenta a aplicação em runtime para correlacionar execução e vulnerabilidades.
SCA	Software Composition Analysis	Analisa componentes de terceiros, dependências e licenças.
SBOM	Software Bill of Materials	Lista de ingredientes do software: componentes, versões e dependências.
CVE	Common Vulnerabilities and Exposures	Identificador público de vulnerabilidade conhecida.
CVSS	Common Vulnerability Scoring System	Pontuação de severidade de vulnerabilidade.
IaC	Infrastructure as Code	Infraestrutura declarada como código, como Terraform, Helm ou YAML de Kubernetes.
SARIF	Static Analysis Results Interchange Format	Formato para trocar resultados de análise estática.
Vault	Cofre de segredos	Sistema para guardar, auditar, versionar e rotacionar segredos.

3. DevOps, DevSecOps e shift-left

DevOps é uma abordagem cultural e operacional que aproxima **desenvolvimento e operação para entregar software com mais frequência, automação e confiabilidade**. DevSecOps adiciona segurança de forma

contínua ao mesmo fluxo. A ideia não é criar uma nova fila burocrática chamada 'segurança', mas inserir controles de segurança em requisitos, código, build, dependências, infraestrutura, deploy e operação.

Em ambiente bancário ou público, DevSecOps não significa que todo desenvolvedor decide sozinho o risco aceitável. Significa que a esteira já incorpora controles, evidências, segregação de papéis, aprovações quando necessárias, rastreabilidade e automação suficiente para reduzir falhas repetitivas.

Pegadinha FGV: DevSecOps não é 'instalar uma ferramenta de SAST'. Ferramenta é meio. DevSecOps envolve cultura, processo, automação, papéis, métricas e controles integrados ao ciclo de vida.

3.1 Shift-left

Shift-left é a antecipação de práticas de segurança para fases iniciais do ciclo de desenvolvimento. Em vez de descobrir vulnerabilidades apenas em homologação ou produção, a equipe tenta detectar problemas no commit, no pull request, no build, na análise de dependências e na revisão de código.

O ponto de prova: **shift-left antecipa controles, mas não elimina controles à direita**. DAST, monitoração, WAF, SIEM, gestão de incidentes, logs, hardening de runtime e resposta continuam relevantes. O candidato erra quando interpreta shift-left como substituição total de segurança em produção.

3.2 Secure by design, secure by default e secure in deployment

No vocabulário moderno de segurança, secure by design indica que a segurança entra na concepção do produto; secure by default indica que a configuração inicial já é segura; secure in deployment indica que a implantação, a infraestrutura e a operação mantêm os controles previstos. Para a FGV, a distinção relevante é evitar colocar toda a responsabilidade no scanner final de pipeline.

Conceito	Ideia	Exemplo em API Java/Quarkus
Security by design	Desenhar com segurança desde requisitos e arquitetura	Definir escopos, perfis, trilha de auditoria e limites de autorização antes de codar
Shift-left	Executar verificações cedo	Secret scan e SAST no pull request
Security gate	Critério automatizado ou manual que bloqueia avanço	Pipeline falha se dependência crítica tiver CVE explorável
Defense in depth	Camadas de controle	Validação no front, autorização no back, logs, rate limit, WAF e monitoração

4. Pipeline DevSecOps no contexto corporativo

Pipeline é a sequência automatizada que transforma alteração de código em artefato testado, empacotado e eventualmente implantado. Em CI/CD, o pipeline é ativo crítico: ele tem acesso a repositórios, credenciais, ambientes, imagens, clusters e secrets. Por isso, a própria esteira precisa ser protegida.

Um fluxo plausível para seu contexto de Angular, Quarkus, APIs e esteiras corporativas seria:

```
Commit/PR
-> lint + testes unitários
-> secret scan
-> SAST no código Angular/TypeScript e Java/Quarkus
-> SCA em package.json, package-lock.json, pom.xml e imagens
-> build do artefato e imagem
-> assinatura/registo imutável do artefato
-> deploy em ambiente de teste/homologação
-> DAST contra aplicação em execução
-> aprovação ou gate de risco
-> deploy controlado
-> logs, métricas, alertas e gestão de vulnerabilidades
```

4.1 Gates de segurança

Gate de segurança é um ponto de decisão. Pode ser bloqueante ou apenas informativo, conforme severidade, maturidade da equipe, criticidade do sistema e política institucional. Em prova, o gate correto normalmente é proporcional ao risco.

Resultado do controle	Decisão razoável	Erro típico
Secret real no repositório	Bloquear, revogar/rotacionar secret e registrar incidente	Apenas apagar a linha do commit
CVE crítica explorável em biblioteca usada	Bloquear ou exigir exceção formal com plano de mitigação	Ignorar porque o build passou
Finding SAST baixo e falso positivo provável	Triar, justificar e acompanhar	Quebrar toda entrega sem análise de risco
DAST encontra endpoint administrativo exposto	Bloquear promoção e corrigir controle de acesso	Tratar como problema apenas de front-end

4.2 Esteira como ambiente sensível

A esteira costuma executar tarefas com identidades de alto privilégio: publicar artefatos, acessar registry, implantar em Kubernetes, consultar cofre de segredos ou abrir mudanças. Logo, aplicar menor privilégio, segregação de ambientes, logs e revisão de permissões na esteira é tão importante quanto proteger a aplicação final.

4.3 Secure SDLC por fase

SDLC - Software Development Life Cycle - é o ciclo de vida de desenvolvimento de software. Em DevSecOps, cada fase recebe controles proporcionais. A banca pode cobrar a técnica adequada para a fase descrita no enunciado.

Fase	Controle típico	Exemplo de prova
Requisitos	Requisitos de segurança, privacidade e critérios de aceite	Exigir auditoria de acesso para dados pessoais
Desenho	Modelagem de ameaças e revisão de arquitetura	Identificar abuso de fluxo, confiança indevida e falta de segregação
Codificação	Revisão de código, SAST, secret scan e padrões seguros	Detectar concatenação insegura de SQL antes do deploy
Build	SCA, geração de SBOM, assinatura e artefato imutável	Bloquear dependência crítica explorável
Teste	DAST, testes autenticados e validação de headers	Testar app rodando em staging
Deploy	IaC scan, controle de permissões e aprovação por risco	Evitar manifesto Kubernetes com privilégio excessivo
Operação	Logs, alertas, gestão de vulnerabilidades e resposta	Detectar abuso de token ou aumento anômalo de erro

4.4 Definition of Ready e Definition of Done

Em times ágeis, DoR - Definition of Ready - e DoD - Definition of Done - podem incorporar critérios de segurança. Um item não deveria entrar em desenvolvimento se ignora requisito crítico de autenticação, autorização ou privacidade. Um item não deveria ser considerado pronto se quebra gate de dependência crítica, expõe secret ou remove log de auditoria obrigatório.

Pegadinha FGV: adicionar segurança à DoD não é burocracia vazia. É transformar controle em critério objetivo de entrega. O erro é tratar segurança como etapa separada apenas após desenvolvimento completo.

5. SAST - Static Application Security Testing

SAST - Static Application Security Testing - analisa código-fonte, bytecode ou artefatos sem executar a aplicação. Ele tenta localizar padrões inseguros, como concatenação de SQL, uso de funções perigosas, exposição de segredo, validação insuficiente, erros de configuração e vulnerabilidades conhecidas por regra.

Em Java/Quarkus, SAST pode apontar uso inseguro de concatenação em consulta, desserialização perigosa, endpoints sem autorização explícita ou tratamento inadequado de exceções. Em Angular/TypeScript, pode apontar uso inseguro de APIs do navegador, manipulação de HTML e práticas que aumentam risco de Cross-Site Scripting - XSS.

5.1 Forças e limites do SAST

Aspecto	SAST
Momento	Cedo: commit, pull request, build
Precisa da aplicação rodando?	Não
Visibilidade	Código e fluxo estático
Bom para	Padrões inseguros, caminhos de dados, uso de API perigosa, falhas simples de validação
Limite	Pode gerar falso positivo e não entende bem lógica de negócio em runtime
Pegadinha	Não garante que a aplicação esteja segura em execução

Ponto FGV: SAST não é 'teste white-box funcional'. É análise de segurança estática. Ele pode até usar visão de código, mas o objetivo é identificar vulnerabilidades ou padrões inseguros, não validar regra de negócio funcional.

6. DAST - Dynamic Application Security Testing

DAST - Dynamic Application Security Testing - **testa a aplicação em execução**. Em geral, opera de fora para dentro, enviando requisições HTTP, simulando entradas maliciosas e verificando respostas, comportamento e exposição. **Ele não precisa conhecer o código-fonte, mas precisa de aplicação implantada em ambiente acessível.**

Em uma API REST - Representational State Transfer -, **DAST pode testar endpoints, parâmetros, autenticação, cabeçalhos** de segurança, **erros** verbosos, **exposição** indevida, injeção e outras falhas observáveis em runtime. **Seu desempenho melhora quando há documentação OpenAPI/Swagger, credenciais de teste e ambiente representativo.**

6.1 Forças e limites do DAST

Aspecto	DAST
Momento	Após deploy em ambiente de teste, homologação ou staging
Precisa da aplicação rodando?	Sim
Visibilidade	Comportamento externo
Bom para	Falhas observáveis em execução, headers, autenticação mal configurada, exposição e injeção
Limite	Cobertura depende de navegação, endpoints conhecidos e dados de teste
Pegadinha	Não vê código morto ou caminho não alcançado pelo teste

6.2 SAST x DAST x IAST

Técnica	Como opera	Momento	Cuidado de prova
SAST	Analisa código sem executar	Antes de deploy	Pode achar padrão inseguro, mas não valida runtime
DAST	Ataca/testa app em execução	Staging/homologação	Não precisa de código, mas depende de cobertura
IAST	Instrumenta app em execução	Testes integrados/runtime	Mistura visibilidade de execução com análise de segurança

7. SCA, dependências e cadeia de suprimentos

SCA - Software Composition Analysis - analisa componentes de terceiros: bibliotecas, frameworks, pacotes NPM, dependências Maven, imagens de contêiner e licenças. A motivação é simples: **aplicações modernas usam muito código de terceiros, e vulnerabilidades podem vir de uma dependência indireta que o time nem chama diretamente no código.**

Em uma aplicação Angular, a análise passa por package.json e lockfiles. Em Quarkus/Maven, passa por pom.xml e árvore de dependências. **Em contêiner, passa pela imagem base e pelos pacotes instalados.** SCA não substitui SAST: SAST olha código próprio; **SCA olha componentes.**

7.1 CVE, CVSS, EPSS e priorização

CVE - Common Vulnerabilities and Exposures - é um identificador público de vulnerabilidade. CVSS - Common Vulnerability Scoring System - ajuda a pontuar severidade. Na vida real, priorizar só por CVSS pode ser insuficiente: é preciso considerar exploração conhecida, exposição, presença efetiva do componente vulnerável, compensações e criticidade do sistema.

7.2 SBOM

SBOM - Software Bill of Materials - é a lista de componentes que compõem o software, com nomes, versões e relações. Para prova, pense em 'lista de ingredientes'. Ela aumenta visibilidade da cadeia de suprimentos, mas não corrige vulnerabilidade sozinha. Sem processo de triagem, atualização e resposta, SBOM vira inventário parado.

Pegadinha FGV: SCA não procura prioritariamente SQL Injection no código próprio; isso é mais típico de SAST/DAST. SCA identifica dependências vulneráveis e riscos de componentes de terceiros.

7.3 Integridade de artefatos e cadeia de suprimentos

Além de conhecer dependências, a organização **precisa saber se o artefato implantado é o mesmo que foi aprovado.** Isso envolve **artefatos imutáveis, versionamento, assinatura, registry controlado e rastreabilidade entre commit, build, imagem e deploy.**

Em prova, a **cadeia de suprimentos aparece quando o risco não está no código de negócio, mas no caminho até a produção:** dependência comprometida, imagem base vulnerável, pacote trocado, credencial de publicação vazada ou pipeline adulterada.

Risco	Controle
Dependência vulnerável	SCA, SBOM, atualização e política de exceção
Artefato adulterado	Assinatura, hash, registry confiável e imutabilidade
Build não rastreável	Vincular commit, pipeline, versão, imagem e aprovação
Imagem base insegura	Base mínima, scan e atualização controlada

8. Secrets, credenciais e vazamento em repositório

Secret é qualquer dado que permite acesso ou assinatura: senha, token, chave de API, client secret OAuth2, chave privada, credencial de banco, certificado, chave de assinatura ou token de serviço. Em DevSecOps, gestão de secrets é ponto central porque pipelines precisam acessar ambientes e serviços.

Regra de prova: **secret vazado deve ser considerado comprometido**. Remover o arquivo do commit mais recente não basta. É preciso revogar ou rotacionar o segredo, investigar uso indevido, revisar logs, reduzir privilégio e impedir novo vazamento com secret scanning e treinamento.

8.1 Boas práticas

Prática	Sentido
Não commitar secrets	Segredo não deve ficar em código, YAML, Dockerfile, README ou log
Usar cofre de segredos	Centralizar armazenamento, rotação, auditoria e acesso granular
Menor privilégio	Pipeline recebe só permissões necessárias, no ambiente necessário
Segredos efêmeros	Preferir tokens curtos e temporários quando possível
Rotação	Trocar segredos periodicamente ou após suspeita/vazamento
Mascaramento em logs	Evitar imprimir token em console de CI/CD

8.2 Configuração x segredo

Nem toda configuração é secret. URL pública de serviço, nome de fila ou flag de feature podem ser configuração. Senha, token e chave privada são secrets. A FGV pode aproximar esses termos: configuração pode estar versionada; segredo, em regra, deve ser protegido em cofre ou mecanismo equivalente.

9. Segurança de IaC, contêineres e ambiente

IaC - Infrastructure as Code - representa infraestrutura por arquivos declarativos, como Terraform, Helm Chart ou manifestos Kubernetes. Como esses arquivos viram ambiente real, erro de configuração vira risco real: bucket público, security group amplo, pod privilegiado, secret em YAML, imagem sem pinagem ou usuário root em contêiner.

Em DevSecOps, é comum escanear IaC e imagens antes do deploy. O objetivo é detectar misconfigurations e vulnerabilidades de base image cedo, mas também manter validação no cluster e em produção.

9.1 Contêineres e imagens

Controle	O que reduz
Imagem mínima	Superfície de ataque e pacotes vulneráveis
Usuário não-root	Impacto de exploração dentro do contêiner
Assinatura de imagem	Risco de artefato adulterado
Registry controlado	Risco de imagem desconhecida
Scan de imagem	Vulnerabilidades em sistema operacional e bibliotecas

10. Desenvolvimento seguro aplicado a APIs

DevSecOps precisa aparecer em decisões concretas de API. Para uma API de pagamentos, por exemplo, não basta que o pipeline compile. É preciso garantir autenticação, autorização, trilha de auditoria, validação de entrada, proteção de segredo, tratamento de erro, rate limit, logging seguro e teste de comportamento em runtime.

10.1 Exemplo prático

Cenário: uma API Quarkus recebe solicitações de pagamento e expõe endpoints para consulta e aprovação. O front-end Angular chama a API com access token emitido por provedor de identidade. A esteira deve verificar:

1. no pull request: testes, SAST, secret scan e revisão de código; 2. no build: SCA de dependências Maven/NPM, geração de artefato e SBOM; 3. no container: scan da imagem, usuário não-root e base image atualizada; 4. em staging: DAST autenticado com usuário de teste, checagem de headers e endpoints; 5. antes da produção: gate de risco, aprovação quando necessário e rastreabilidade; 6. após deploy: logs, métricas e alertas.

A FGV pode trocar o ponto de controle: pedir SAST quando a descrição é claramente DAST, ou pedir apagar secret quando o correto é rotacionar. Seu ganho está em identificar o momento da esteira e o tipo de visibilidade da técnica.

11. Observabilidade, feedback e segurança após o deploy

DevSecOps não termina no deploy. A operação precisa devolver sinais ao ciclo de desenvolvimento: alertas de exploração, falhas recorrentes, abuso de endpoints, erro de autorização, uso de secrets, anomalias de tráfego e vulnerabilidades recém-divulgadas em dependências já publicadas.

Essa retroalimentação fecha o ciclo: achados de produção geram correções, testes de regressão, novas regras de SAST/DAST, atualização de dependências ou mudança arquitetural. Para a FGV, isso reforça que **DevSecOps é contínuo, não uma verificação pontual.**

11.1 Métricas úteis

Métrica	O que indica
Tempo para corrigir vulnerabilidade	Eficiência de resposta a risco técnico
Percentual de builds bloqueados por achado crítico	Saúde dos controles e qualidade do código
Falsos positivos por ferramenta	Necessidade de ajuste de regras
Dependências com CVE sem plano	Risco de supply chain acumulado
Secrets detectados por período	Maturidade de gestão de credenciais

12. Tabela-mestra de distinções FGV

Par de conceitos	Diferença que derruba candidato
SAST x DAST	SAST analisa código parado; DAST testa aplicação rodando.
SAST x SCA	SAST olha código próprio; SCA olha dependências e componentes.
DAST x pentest	DAST é automação recorrente; pentest é avaliação mais exploratória e humana.
Secret scan x vault	Secret scan detecta vazamento; vault armazena e gerencia segredo.
CI x CD	CI integra e testa; CD promove/implanta artefato.
Delivery x Deployment	Delivery prepara para produção com decisão manual; deployment automatiza a implantação.
Shift-left x segurança total	Shift-left antecipa controles, mas não substitui runtime, monitoração e resposta.
SBOM x correção	SBOM dá visibilidade; correção exige gestão de vulnerabilidades e atualização.

Par de conceitos	Diferença que derruba candidato
Gate x checklist manual	Gate é ponto formal de decisão; pode ser automatizado ou manual com evidência.

13. Pegadinhas recorrentes para a FGV

1. Dizer que DAST analisa código-fonte. Errado: DAST testa aplicação em execução.
2. Dizer que SAST detecta melhor toda falha de lógica de negócio. Cuidado: falhas dependentes de contexto de execução podem escapar da análise estática.
3. Tratar SCA como sinônimo de SAST. Errado: SCA é dependência/componente.
4. Achar que shift-left elimina testes posteriores. Errado: antecipa, mas não substitui controles em staging/produção.
5. Achar que apagar secret do repositório resolve vazamento. Errado: deve revogar/rotacionar e investigar.
6. Achar que DevSecOps é função exclusiva do time de segurança. Errado: segurança é responsabilidade compartilhada, com governança e automação.
7. Confundir pipeline seguro com aplicação segura. Um pipeline inseguro pode comprometer aplicação segura; uma aplicação segura também pode ser entregue por esteira vulnerável.
8. Confundir vulnerabilidade em dependência com vulnerabilidade explorável no sistema. Nem toda CVE em árvore de dependência é explorável, mas precisa triagem baseada em risco.

14. Checklist final do que memorizar

- DevSecOps = segurança integrada ao DevOps, não fase final de auditoria.
- Shift-left = antecipar controles; não eliminar validações à direita.
- SAST = código sem execução; DAST = aplicação rodando; IAST = runtime instrumentado.
- SCA = dependências, componentes, licenças, CVEs e SBOM.
- Secret vazado = secret comprometido: rotacionar, revogar, auditar e corrigir causa.
- Pipeline CI/CD é ativo sensível e alvo de ataque; precisa menor privilégio, logs e hardening.
- Gates devem ser proporcionais ao risco; bloquear tudo sem triagem pode ser ruim, liberar crítico sem mitigação é pior.
- SBOM aumenta visibilidade, mas não substitui correção nem gestão de vulnerabilidades.

15. Lista de questões autorais - padrão FGV

Instruções: resolva as 20 questões antes de consultar o gabarito. As questões misturam conceitos, casos práticos, afirmativas, V/F, associação e interpretação de cenário. Não há pista de gabarito nesta seção.

1. Em uma esteira de desenvolvimento de um sistema interno do TJSC, a equipe deseja identificar, ainda no pull request, padrões de código potencialmente inseguros em uma aplicação Java/Quarkus, sem precisar implantar a aplicação em ambiente de homologação. A técnica mais adequada é:

- (A) Dynamic Application Security Testing - DAST, por simular ataques contra a aplicação em execução.
- (B) Static Application Security Testing - SAST, por analisar código ou artefatos sem executar a aplicação.
- (C) Software Composition Analysis - SCA, por validar exclusivamente regras de autorização em runtime.
- (D) Runtime Application Self-Protection - RASP, por revisar dependências de terceiros antes do build.
- (E) Teste de carga, por identificar padrões de código vulnerável em tempo de compilação.

2. Uma equipe DevSecOps decide adotar shift-left security em um projeto Angular + Quarkus. Considerando o sentido técnico da prática, assinale a alternativa correta.

- (A) Shift-left consiste em deslocar todos os controles para produção, pois o ambiente real é o único capaz de revelar vulnerabilidades.
- (B) Shift-left elimina a necessidade de DAST, monitoração e resposta a incidentes, desde que o SAST seja executado no commit.
- (C) Shift-left significa antecipar atividades de segurança para fases iniciais do ciclo, sem dispensar validações posteriores.
- (D) Shift-left é sinônimo de criptografar todo o tráfego entre microsserviços com TLS.
- (E) Shift-left corresponde à aprovação manual obrigatória de toda mudança, antes da abertura de pull request.

3. No contexto de DevSecOps, analise as afirmativas a seguir. I. SCA permite identificar componentes de terceiros vulneráveis ou com risco de licença. II. SAST e SCA são equivalentes, pois ambos procuram vulnerabilidades apenas em bibliotecas externas. III. Um SBOM aumenta a visibilidade sobre componentes usados pelo software, mas não corrige vulnerabilidades automaticamente. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) II e III, apenas.
- (D) I, II e III.
- (E) I e III, apenas.

4. Durante uma varredura, uma ferramenta acessa uma aplicação web já implantada em ambiente de homologação, envia requisições HTTP com entradas maliciosas e observa respostas para identificar falhas exploráveis. Essa ferramenta é classificada, predominantemente, como:

- (A) SAST.
- (B) SCA.
- (C) SBOM.
- (D) Secret scanning.
- (E) DAST.

5. Um token de acesso a ambiente de homologação foi encontrado em um commit já enviado ao repositório remoto. A providência mais adequada, sob a ótica de DevSecOps, é:

- (A) Remover a linha no próximo commit, mantendo o token ativo para evitar indisponibilidade.
- (B) Abrir uma issue de baixa prioridade, pois ambiente de homologação não tem relevância de segurança.
- (C) Revogar ou rotacionar o token, investigar uso, registrar evidência e corrigir a causa do vazamento.
- (D) Mover o token para outro arquivo versionado com nome menos explícito.
- (E) Codificar o token em Base64 antes de novo commit, pois isso elimina o risco de leitura.

6. Relacione as técnicas aos seus focos principais. 1. SAST 2. DAST 3. SCA 4. Secret scanning () Aplicação em execução. () Código ou artefato sem execução. () Dependências e componentes de terceiros. () Chaves, senhas e tokens expostos. A sequência correta é:

- (A) 2 - 1 - 3 - 4.
- (B) 1 - 2 - 4 - 3.
- (C) 2 - 3 - 1 - 4.
- (D) 3 - 1 - 2 - 4.
- (E) 4 - 2 - 3 - 1.

7. Uma organização implementou pipeline CI/CD que compila o código, executa testes unitários, cria imagem de contêiner e faz deploy automático em produção usando uma credencial com permissão administrativa ampla. O principal problema de segurança está em:

- (A) usar testes unitários, pois eles devem ser substituídos por DAST.
- (B) usar contêiner, pois DevSecOps é incompatível com Docker.
- (C) automatizar etapas, pois CI/CD deve ser sempre manual.
- (D) concentrar privilégios elevados na esteira, ampliando impacto caso a pipeline seja comprometida.
- (E) executar build antes do deploy, pois isso dificulta rastreabilidade.

8. Sobre Continuous Delivery e Continuous Deployment, assinale a alternativa correta.

- (A) Continuous Delivery normalmente deixa o artefato pronto para produção, podendo haver decisão manual de liberação; Continuous Deployment automatiza também a implantação.
- (B) Continuous Deployment é sempre menos automatizado que Continuous Delivery.
- (C) Continuous Delivery exige que o código seja entregue apenas uma vez por mês para reduzir risco.
- (D) Ambos significam exclusivamente testes unitários em máquina local do desenvolvedor.
- (E) Continuous Delivery é sinônimo de DAST, enquanto Continuous Deployment é sinônimo de SAST.

9. Em uma API REST do tribunal, o DAST não acessou endpoints protegidos porque não recebeu credenciais de teste nem descrição OpenAPI/Swagger. A conclusão tecnicamente mais adequada é:

- (A) o DAST sempre garante cobertura completa, mesmo sem autenticação.
- (B) a ausência de achados prova que a API não possui vulnerabilidades.
- (C) a cobertura do DAST pode ter sido limitada, exigindo configuração, autenticação e escopo adequados.
- (D) o DAST deve ser substituído por SBOM, pois ambos têm o mesmo objetivo.
- (E) o problema demonstra que DAST analisa apenas código-fonte.

10. Assinale a alternativa que apresenta uma prática compatível com gestão segura de secrets em pipelines.

- (A) Armazenar secrets em arquivo YAML versionado, desde que o repositório seja privado.
- (B) Usar cofre de segredos, aplicar menor privilégio, auditar acesso e rotacionar credenciais quando necessário.
- (C) Imprimir token no log da pipeline para facilitar auditoria humana.
- (D) Compartilhar a mesma senha administrativa entre todos os projetos para simplificar suporte.
- (E) Usar Base64 como técnica de criptografia forte para proteger credenciais.

11. Considere as afirmativas sobre gates de segurança em DevSecOps. I. Um gate pode bloquear a promoção de uma versão quando uma vulnerabilidade crítica explorável é encontrada. II. Todo finding de baixa severidade deve bloquear automaticamente produção, sem triagem e sem política de exceção. III. Gates podem ser automatizados, mas a decisão de risco pode depender de política institucional e criticidade do sistema. Está correto o que se afirma em:

- (A) I e III, apenas.
- (B) I e II, apenas.
- (C) II e III, apenas.
- (D) III, apenas.
- (E) I, II e III.

12. Uma ferramenta aponta CVE crítica em biblioteca transitiva de uma aplicação, mas a equipe ainda precisa verificar se a versão vulnerável está empacotada, se o componente é usado e se há exposição real. Essa situação ilustra melhor:

- (A) a inutilidade de SCA, pois vulnerabilidades em dependências nunca são exploráveis.
- (B) a necessidade de triagem de vulnerabilidades baseada em contexto e risco.
- (C) a obrigatoriedade de desligar a aplicação antes de qualquer análise.
- (D) a substituição do SCA por teste de usabilidade.
- (E) a prova de que CVSS não possui utilidade alguma.

13. Sobre IaC - Infrastructure as Code - em DevSecOps, assinale a opção correta.

- (A) Arquivos Terraform, Helm e manifestos Kubernetes podem ser analisados para identificar configurações inseguras antes do deploy.
- (B) IaC elimina a necessidade de controle de acesso, pois a infraestrutura passa a ser declarativa.
- (C) IaC scanning procura apenas erros de ortografia em comentários.
- (D) IaC é incompatível com pipelines CI/CD e deve ser executada manualmente.
- (E) Misconfiguration em IaC não gera risco real, pois arquivos declarativos não afetam ambientes.

14. Em uma aplicação Quarkus empacotada em imagem de contêiner, a equipe deseja reduzir superfície de ataque e impacto de exploração. Qual conjunto de práticas é mais adequado?

- (A) Usar imagem base mínima, usuário não-root, scan de imagem e registry controlado.
- (B) Executar sempre como root, para evitar falhas de permissão em produção.
- (C) Instalar ferramentas administrativas desnecessárias na imagem final, para facilitar troubleshooting.
- (D) Desabilitar logs de segurança, pois logs aumentam consumo de disco.
- (E) Publicar imagem manualmente em registry pessoal do desenvolvedor.

15. Em relação ao DevSecOps, assinale a alternativa INCORRETA.

- (A) DevSecOps busca integrar segurança ao ciclo de desenvolvimento e operação.
- (B) Automação de segurança pode incluir SAST, SCA, secret scanning, IaC scanning e DAST.
- (C) O pipeline CI/CD deve ser tratado como ativo sensível, pois pode ter acesso a credenciais e ambientes.
- (D) DevSecOps significa transferir toda responsabilidade de segurança para uma ferramenta de scanner.
- (E) A priorização de vulnerabilidades deve considerar severidade, exposição, explorabilidade e criticidade.

16. Um sistema público trata dados pessoais em ambiente de homologação. Para testes automatizados, a equipe decide usar cópia integral da produção sem mascaramento, porque o ambiente é interno. À luz de DevSecOps e proteção de dados, a decisão é:

- (A) adequada, pois ambiente interno elimina risco de privacidade.
- (B) adequada se o DAST for executado depois da cópia.
- (C) inadequada; deve-se avaliar uso de dados sintéticos, mascaramento, minimização e controles de acesso.
- (D) obrigatória, pois testes só são válidos com dados reais completos.
- (E) irrelevante para segurança, pois dados de teste não pertencem ao SDLC.

17. Analise o trecho de pipeline abaixo. - npm install - mvn package - docker build - kubectl apply Não há verificação de dependências, secrets, imagem, IaC ou teste de segurança. O diagnóstico mais adequado é:

- (A) a pipeline é suficiente, porque DevSecOps se limita ao deploy automatizado.
- (B) há automação de entrega, mas faltam controles de segurança integrados ao fluxo.
- (C) a pipeline é inválida porque usa duas linguagens no mesmo projeto.
- (D) SAST e DAST são impossíveis em projetos com contêiner.
- (E) o uso de kubectl elimina necessidade de governança.

18. Ao revisar findings de SAST, a equipe identifica muitos alertas que não se confirmam após análise. A reação mais madura é:

- (A) desabilitar definitivamente toda análise estática.
- (B) ajustar regras, classificar falsos positivos, definir baseline e manter triagem baseada em risco.
- (C) migrar todos os achados automaticamente para produção.
- (D) substituir SAST por SBOM, pois SBOM analisa código próprio.
- (E) ignorar SAST sempre que houver testes unitários.

19. Qual alternativa representa melhor a relação entre SBOM e gestão de vulnerabilidades?

- (A) SBOM substitui patching, pois inventário corrige vulnerabilidades automaticamente.
- (B) SBOM é útil para visibilidade de componentes, mas precisa ser associado a monitoramento, análise e correção de vulnerabilidades.
- (C) SBOM é uma técnica de criptografia de secrets.
- (D) SBOM é um tipo de DAST autenticado.
- (E) SBOM impede qualquer ataque de supply chain mesmo sem processo de atualização.

20. Em uma questão sobre DevSecOps, a FGV descreve: 'a ferramenta instrumenta a aplicação em execução e correlaciona entradas com fluxo interno para reduzir falsos positivos'. A técnica descrita é mais próxima de:

- (A) SCA.
- (B) Secret scanning.
- (C) SBOM.
- (D) Linting puramente sintático.
- (E) IAST.

16. Gabarito resumido

1-B 2-C 3-E 4-E 5-C
6-A 7-D 8-A 9-C 10-B
11-A 12-B 13-A 14-A 15-D
16-C 17-B 18-B 19-B 20-E

O gabarito foi definido após a construção e auditoria das alternativas, sem ciclo visual deliberado.

17. Comentários completos

Questão 1 - Gabarito: B

Correta. SAST analisa código ou artefatos sem executar a aplicação, exatamente o caso do pull request.

- (A) **Errada.** DAST exige aplicação rodando e simula ataques em runtime.
- (C) **Errada.** SCA trata dependências e componentes, não valida exclusivamente autorização em runtime.
- (D) **Errada.** RASP atua dentro da aplicação em execução, não revisa dependências antes do build.
- (E) **Errada.** Teste de carga mede desempenho, não padrões de código vulnerável.

Questão 2 - Gabarito: C

Correta. Shift-left antecipa controles de segurança, mas não elimina validações posteriores.

- (A) **Errada.** Deslocar tudo para produção é o oposto da antecipação.
- (B) **Errada.** SAST no commit ajuda, mas não substitui DAST, monitoração e resposta.
- (D) **Errada.** TLS é controle de comunicação, não definição de shift-left.
- (E) **Errada.** Aprovação manual antes do PR não resume shift-left.

Questão 3 - Gabarito: E

Correta. I e III estão corretas: SCA olha componentes e SBOM dá visibilidade sem corrigir sozinho.

- (A) **Errada.** Está incompleta, pois III também está correta.
- (B) **Errada.** II está errada: SAST não se limita a bibliotecas externas.
- (C) **Errada.** II está errada, embora III esteja correta.
- (D) **Errada.** II torna o conjunto incorreto.

Questão 4 - Gabarito: E

Correta. Aplicação em execução testada por requisições externas caracteriza DAST.

(A) Errada. SAST não executa a aplicação.

(B) Errada. SCA analisa dependências.

(C) Errada. SBOM é inventário de componentes.

(D) Errada. Secret scanning procura segredos expostos.

Questão 5 - Gabarito: C

Correta. Secret vazado deve ser revogado/rotacionado, investigado e tratado na causa raiz.

(A) Errada. Apagar linha não invalida token já exposto.

(B) Errada. Homologação também pode ter dados e acessos relevantes.

(D) Errada. Mover para outro arquivo versionado mantém o vazamento.

(E) Errada. Base64 é codificação, não proteção criptográfica suficiente.

Questão 6 - Gabarito: A

Correta. DAST = aplicação rodando; SAST = código sem execução; SCA = dependências; secret scanning = segredos.

(B) Errada. Inverte SAST e DAST e troca dependências por secrets.

(C) Errada. Coloca SCA onde deveria estar SAST.

(D) Errada. Começa com SCA, mas a primeira descrição é DAST.

(E) Errada. Começa com secrets, mas a primeira descrição é aplicação em execução.

Questão 7 - Gabarito: D

Correta. Credencial ampla na pipeline aumenta impacto de comprometimento; esteira é ativo sensível.

(A) Errada. Testes unitários não substituem DAST, mas não são o problema principal.

(B) Errada. DevSecOps é compatível com contêineres.

(C) Errada. Automação é desejável; o problema é privilégio excessivo.

(E) Errada. Build antes de deploy é prática normal e melhora rastreabilidade.

Questão 8 - Gabarito: A

Correta. Delivery prepara/libera com possível aprovação manual; deployment automatiza a implantação.

(B) Errada. Deployment tende a ser mais automatizado, não menos.

(C) Errada. Frequência mensal não define entrega contínua.

(D) Errada. CI/CD não se limita a testes locais.

(E) Errada. Não há equivalência entre CD e SAST/DAST.

Questão 9 - Gabarito: C

Correta. DAST depende de escopo, autenticação, navegação e endpoints conhecidos; ausência de achado não prova segurança.

(A) Errada. DAST não garante cobertura completa automaticamente.

(B) Errada. Sem cobertura adequada, conclusão é fraca.

(D) Errada. SBOM não substitui DAST.

(E) Errada. DAST não analisa código-fonte; testa runtime.

Questão 10 - Gabarito: B

Correta. Cofre, menor privilégio, auditoria e rotação são práticas adequadas.

(A) Errada. Repositório privado não é cofre de segredos.

(C) Errada. Logs não devem revelar tokens.

(D) Errada. Senha compartilhada viola menor privilégio e responsabilização.

(E) Errada. Base64 não é criptografia forte.

Questão 11 - Gabarito: A

Correta. I e III estão corretas; II é exagero, pois bloqueio automático de todo baixo risco sem triagem pode ser contraproducente.

(B) Errada. II está errada.

(C) Errada. I também está correta, mas II errada.

(D) Errada. III está correta, mas I também.

(E) Errada. II torna o conjunto incorreto.

Questão 12 - Gabarito: B

Correta. SCA gera achado que precisa triagem: presença, uso, exposição, explorabilidade e criticidade.

(A) Errada. Dependências podem ser exploráveis; não são irrelevantes.

(C) Errada. Desligar aplicação não é requisito de análise.

(D) Errada. Teste de usabilidade não substitui SCA.

(E) Errada. CVSS é útil, apenas não é suficiente sozinho.

Questão 13 - Gabarito: A

Correta. IaC scanning analisa arquivos de infraestrutura para detectar misconfigurations antes do deploy.

(B) Errada. Infra declarativa ainda exige controle de acesso.

(C) Errada. Não é correção ortográfica; é segurança de configuração.

(D) Errada. IaC é altamente compatível com CI/CD.

(E) Errada. Arquivos declarativos afetam ambientes reais.

Questão 14 - Gabarito: A

Correta. Imagem mínima, usuário não-root, scan e registry controlado reduzem superfície e impacto.

(B) Errada. Root aumenta impacto de exploração.

(C) Errada. Ferramentas desnecessárias ampliam superfície.

(D) Errada. Logs de segurança são importantes para detecção e auditoria.

(E) Errada. Registry pessoal dificulta governança e rastreabilidade.

Questão 15 - Gabarito: D

Correta. DevSecOps não transfere toda segurança a uma ferramenta; é processo, cultura, automação e governança.

(A) Errada. Está correta, portanto não é a incorreta.

(B) Errada. Está correta: esses controles são típicos.

(C) Errada. Está correta: pipeline tem credenciais e poder de deploy.

(E) Errada. Está correta: priorização deve considerar contexto.

Questão 16 - Gabarito: C

Correta. Dados pessoais em testes exigem minimização, mascaramento/sintéticos e controle de acesso quando aplicável.

(A) Errada. Ambiente interno não elimina risco.

(B) Errada. DAST não resolve uso indevido de dados reais.

(D) Errada. Testes podem usar dados sintéticos ou mascarados.

(E) Errada. Dados de teste fazem parte do SDLC e do risco.

Questão 17 - Gabarito: B

Correta. A pipeline automatiza entrega, mas falta integrar controles de segurança.

- (A) **Errada.** DevSecOps não se limita a deploy.
- (C) **Errada.** Projetos full stack frequentemente usam mais de uma tecnologia.
- (D) **Errada.** Contêiner não impede SAST/DAST.
- (E) **Errada.** kubectl não elimina governança.

Questão 18 - Gabarito: B

Correta. Maturidade envolve ajustar regras, baselines, triagem e risco, não desligar o controle.

- (A) **Errada.** Desabilitar tudo perde valor preventivo.
- (C) **Errada.** Achado não deve ir para produção automaticamente.
- (D) **Errada.** SBOM não analisa código próprio como SAST.
- (E) **Errada.** Testes unitários não substituem SAST.

Questão 19 - Gabarito: B

Correta. SBOM dá visibilidade, mas precisa de processo de monitoramento, análise e correção.

- (A) **Errada.** Inventário não corrige vulnerabilidade.
- (C) **Errada.** SBOM não é criptografia de secrets.
- (D) **Errada.** SBOM não é DAST.
- (E) **Errada.** Sem atualização e resposta, SBOM não impede todos ataques.

Questão 20 - Gabarito: E

Correta. IAST instrumenta a aplicação em execução e correlaciona runtime com análise de segurança.

- (A) **Errada.** SCA olha componentes.
- (B) **Errada.** Secret scanning procura segredos.
- (C) **Errada.** SBOM inventaria componentes.
- (D) **Errada.** Linting sintático não instrumenta runtime.

18. Referências consultadas

As fontes abaixo foram usadas para calibrar escopo, terminologia técnica e estilo de cobrança. As questões são autorais e não copiam questões reais.

- TJSC - Edital n. 10/2026, retificado em 10/04/2026.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV - Assembleia Legislativa do Amazonas - Analista Legislativo - Analista de Sistema, prova 2025, questão sobre DAST e SAST.
- FGV - CGE-SP - Auditor Estadual de Controle - Tecnologia da Informação, prova 2025, item sobre SAST, DAST, IAST e fuzzing.
- NIST SP 800-218 - Secure Software Development Framework - SSDF, versão 1.1.
- OWASP DevSecOps Guideline.
- OWASP CI/CD Security Cheat Sheet.
- OWASP Secrets Management Cheat Sheet.
- OWASP Top 10 CI/CD Security Risks.